Computation Theory

The 80/20 Revision Guide

Part IB — Cambridge CST — Paper 6

How this guide is organised

Computation Theory sets two questions a year on **Paper 6**. Across **2018–2025 (16 questions)**:

Topic	Appearances	Share
λ-calculus & λ-definability	6+6	38% (each tag)
Register machines	5	31%
Computability / (un)decidability / r.e. sets	4	25%
Primitive & partial recursive functions	2	12%
Church–Turing thesis; fixed-point theorems	2 each	12%
Turing machines; Gödel numbering	1 each	6%

80/20 verdict. λ-calculus is the single biggest theme — it shows up almost every year as "β-reduce this", "show f is λ-definable", or "what is the **Y** combinator". **Register machines** are next (program them; the universal machine; the halting proof). Then **computability** (undecidability by diagonalisation / reduction). **Turing machines barely appear (6%)** — know the definition and that they're *equivalent* to register machines, but don't over-invest. Everything hangs off one fact: **all the models compute the same functions** — so the course is really about that single notion of "computable" and its limits. Write *definitions first*; these questions reward precision.

---

1 The one big idea: equivalent models of computation

Exam-critical

The course introduces several formalisations of "algorithm" and proves them **all equivalent**:

$$\text{register-machine computable} = \text{Turing computable} = \text{partial recursive} = \lambda\text{-definable}.$$

This shared class is what we call **computable**. The **Church–Turing thesis** (not a theorem — it can't be, "algorithm" being informal) asserts: *every intuitively-computable function is in this class.*

The payoff is undecidability: because programs can be **coded as data** (Gödel numbering), a single *universal* machine can run any program, and a diagonal argument then exhibits problems *no* program can solve — the **Halting Problem**, and via it Hilbert's *Entscheidungsproblem* and Hilbert's 10th Problem.

2 λ-calculus (Tier 1 — the spine)

Exam-critical

[title=Syntax, alpha, beta] **λ-terms:** variables x; **abstraction** $(\lambda x.M)$; **application** $(M\,M')$. Application is left-assoc; $\lambda x\,y.M$ abbreviates $\lambda x.\lambda y.M$.

Free/bound: λx binds x in its body; $FV(\lambda x.M) = FV(M) \setminus \{x\}$. A **combinator** is a closed term, e.g. $\mathrm{I} = \lambda x.x$, $\mathrm{K} = \lambda x\,y.x$.

α-equivalence: rename bound variables $(\lambda x.x =_\alpha \lambda y.y)$ — terms equal up to bound names.

β-reduction: a redex $(\lambda x.M)\,N \to_\beta M[N/x]$ (capture-avoiding substitution). $\twoheadrightarrow_\beta$ is many-step; $=_\beta$ adds expansion (symmetry).

Exam trap

Substitution is capture-avoiding. Before substituting N for x in M, α-rename so no free variable of N is captured by a binder in M. E.g. $(\lambda y.x)[y/x]$ must first rename: $=_\alpha (\lambda z.x)[y/x] = \lambda z.y$, *not* $\lambda y.y$.

Worth knowing

Normal forms & confluence. A term is in **β-normal form** if it has no redex. **Church–Rosser:** $\twoheadrightarrow_\beta$ is *confluent* — if $M \twoheadrightarrow_\beta M_1$ and $M \twoheadrightarrow_\beta M_2$ then both reduce to a common M'. Consequence: a β-normal form is **unique up to** α if it exists. Some terms have none: $\Omega = (\lambda x.x\,x)(\lambda x.x\,x) \to_\beta \Omega$ loops forever. **Normal-order** (leftmost-outermost) reduction *always* reaches the normal form if one exists — so reduction order matters: $(\lambda x.y)\,\Omega \to_\beta y$ if you reduce the outer redex first, but loops if you reduce Ω.

2.1 Encoding data (memorise these)

Exam-critical

[title=Church numerals and basic combinators]

$$\underline{n} \equiv \lambda f\,x.\ \underbrace{f(\cdots(f}_{n}\ x)\cdots) = \lambda f\,x.f^n x \qquad \mathbf{Succ} \equiv \lambda n\,f\,x.\ f\,(n\,f\,x)$$

$$\mathbf{True} \equiv \lambda x\,y.x, \quad \mathbf{False} \equiv \lambda x\,y.y, \quad \mathbf{If} \equiv \lambda f\,x\,y.f\,x\,y$$

$$\mathbf{Pair} \equiv \lambda x\,y\,f.f\,x\,y, \quad \mathbf{Fst} \equiv \lambda p.p\,\mathbf{True}, \quad \mathbf{Snd} \equiv \lambda p.p\,\mathbf{False}$$

Note $\mathbf{False} =_\alpha \underline{0}$. Addition is $\lambda m\,n\,f\,x.\,m\,f\,(n\,f\,x)$.

Worked: Succ of 1 reduces to 2

$\mathbf{Succ}\,\underline{1} = (\lambda nfx.f(nfx))(\lambda fx.fx) \twoheadrightarrow_\beta \lambda fx.f((\lambda fx.fx)fx) \twoheadrightarrow_\beta \lambda fx.f(fx) = \underline{2}$. And $\mathbf{Fst}(\mathbf{Pair}\,M\,N) \twoheadrightarrow_\beta (\lambda f.fMN)\mathbf{True} \twoheadrightarrow_\beta \mathbf{True}\,M\,N \twoheadrightarrow_\beta M$.

2.2 Recursion via the fixed-point combinator

Exam-critical

λ-calculus has no built-in recursion — you get it from a **fixed-point combinator**. For *every* term M there is a y with $M\,y =_\beta y$. **Curry's** combinator:

$$\mathbf{Y} \equiv \lambda f.\big(\lambda x.f(x\,x)\big)\big(\lambda x.f(x\,x)\big), \qquad \mathbf{Y}\,M =_\beta M\,(\mathbf{Y}\,M).$$

So a recursive definition $h = \Phi(h)$ is solved by $h = \mathbf{Y}\,\Phi$ — this is how primitive recursion and minimisation are encoded in λ-calculus.

2.3 λ-definability (the 38% definition — get it exact)

Exam-critical

$f \in \mathbb{N}^n \rightharpoonup \mathbb{N}$ is **λ-definable** if some closed term F *represents* it: for all $\vec{x}$,

- if $f(\vec{x}) = y$ then $F\,\underline{x_1}\cdots\underline{x_n} =_\beta \underline{y}$;
- if $f(\vec{x})\uparrow$ (undefined) then $F\,\underline{x_1}\cdots\underline{x_n}$ has **no β-normal form**.

Theorem. A partial function is **computable iff λ-definable**. (Proof skeleton: every partial recursive function is λ-definable — basic functions, composition, primitive recursion via $\mathbf{Y}$, minimisation; and λ-definable $\Rightarrow$ register-machine computable.)

Exam trap

The *undefined* case is the subtle one: you must ensure the term has **no normal form** when f is undefined (not just that it returns something odd). This is why composing partial functions needs care — force each argument sub-term to fully reduce (e.g. apply $\mathrm{I}\,\mathrm{I}$) so that a non-terminating argument makes the whole term non-terminating.

3 Register machines (Tier 1)

Exam-critical

[title=Definition] Finitely many registers $R_0, R_1, \ldots$ (each holds a natural number) and a program: a finite list of labelled instructions, each of the form

$$\begin{aligned} R^+ \to L' &\;:\; \text{add 1 to } R, \text{ goto } L';\\ R^- \to L', L'' &\;:\; \text{if } R{>}0 \text{ subtract 1, goto } L'; \text{ else goto } L'';\\ \mathsf{HALT} &\;:\; \text{stop.} \end{aligned}$$

A **configuration** is $(\ell, r_0, \ldots, r_n)$ (current label + register contents). Computation is the deterministic sequence of configurations; it **halts** at HALT or at a label with no instruction. Determinism $\Rightarrow$ a program computes a **partial function**.

Worth knowing

Programming idioms. You can't read a register without emptying it, so **copying S to D** needs a temp T: empty D; decrement S while incrementing both D and T; then move T back to S. Graphical notation: one node per instruction, START $\to [L_0]$, arrows for jumps (R^- has two out-arrows). Be ready to (a) trace a given machine, (b) write one for a small function (addition,

copy, multiplication).

Worth knowing

Coding & the universal machine. Encode pairs/lists/programs as single numbers (**Gödel numbering**, e.g. $\langle x, y\rangle = 2^x(2y+1)$, lists by nested pairs). Then a program is a number e, and there is a **universal register machine** U that, given e and (coded) input, simulates program e — i.e. computes φ_e, the function of the e-th program. "Code as data" is the whole basis of undecidability.

4 Computability & undecidability (Tier 1/2)

Exam-critical

[title=Decidable, undecidable, r.e.] Fix an enumeration $\varphi_0, \varphi_1, \ldots$ of all computable (1-argument) partial functions.

- A set $S \subseteq \mathbb{N}$ is **decidable** if its characteristic function is computable (an algorithm answers "$x \in S$?" and always halts).
- S is **recursively enumerable (r.e.)** if it is the domain of a computable partial function — a machine halts on exactly the members of S.
- S is decidable iff both S and its complement are r.e.

Exam-critical

[title=The Halting Problem is undecidable] There is no total computable H with $H(A, D) = 1$ iff program A halts on input D. **Proof (diagonalisation).** Suppose H exists. Build

$$C: \text{ "on input } A, \text{ compute } H(A,A); \text{ if } H(A,A) = 0 \text{ return } 1, \text{ else loop forever."}$$

Then $C(A)\downarrow \iff H(A,A) = 0 \iff A(A)\uparrow$. Apply C to its own code C: $C(C)\downarrow \iff C(C)\uparrow$ — contradiction. So H cannot exist.

Worth knowing

Undecidability by reduction. To show a new problem P undecidable, *reduce* the Halting Problem to it: show a decider for P would give a decider for Halting. The same trick (via coding Halting instances as arithmetic statements $\Phi_{A,D}$ with "$\Phi_{A,D}$ provable $\iff$ $A(D)\downarrow$") settles Hilbert's *Entscheidungsproblem*; Hilbert's 10th (Diophantine equations) fell the same way (Matiyasevich, via register machines). An **uncomputable function** also drops straight out: $g(x) = \varphi_x(x) + 1$ if $\varphi_x(x)\downarrow$ can't be computable (it would differ from every φ_x).

5 Recursive functions (Tier 2)

Exam-critical

Primitive recursive (PRIM) = the smallest class containing the *basic functions* — zero **0**, successor succ, projections proj_i^n — and closed under **composition** and **primitive recursion**:

$$h(\vec{x}, 0) = f(\vec{x}), \qquad h(\vec{x}, y{+}1) = g(\vec{x}, y, h(\vec{x}, y)).$$

Partial recursive = PRIM closed also under **minimisation** $\mu y[\, f(\vec{x}, y) = 0\,]$ (search for the

least y — may not halt ⇒ partial). **Partial recursive = computable.**

Worth knowing

Ackermann's function is *total* and computable but **not primitive recursive** — it grows faster than any p.r. function. So PRIM $\subsetneq$ total computable: primitive recursion alone can't capture all algorithms; minimisation (unbounded search) is what adds the missing power (and the possibility of non-termination).

6 Turing machines & Church–Turing (Tier 3 — low yield)

Worth knowing

A **Turing machine** has a two-way infinite tape, a head that reads/writes one cell and moves L/R, and a finite control. **Turing computable = register-machine computable** (each simulates the other), which is the evidence for the **Church–Turing thesis**. This course de-emphasises TMs (only ∼6% of questions) in favour of register machines and λ-calculus — know the definition and the equivalence, and move on.

Exam checklist

Exam-critical

1. **Equivalence & Church–Turing:** RM = TM = partial recursive = λ-definable = "computable"; the thesis is not provable.
2. **λ-calculus:** α/β, *capture-avoiding* substitution, normal form, Church–Rosser (unique NF), normal-order reduction, Ω non-termination. Encodings: $\underline{n} = \lambda f x.f^n x$, **Succ**, booleans, **Pair/Fst/Snd**. $\mathbf{Y}\,M =_\beta M(\mathbf{Y}\,M)$.
3. **λ-definability:** $F\,\underline{\vec{x}} =_\beta \underline{f(\vec{x})}$ when defined, *no β-NF* when undefined; computable $\iff$ λ-definable.
4. **Register machines:** instructions $R^+ \to L'$, $R^- \to L', L''$, HALT; configurations; copy-via-temp idiom; coding programs as numbers; universal machine computes φ_e.
5. **Undecidability:** decidable vs r.e. (decidable $\iff$ S and $\overline{S}$ both r.e.); the Halting diagonal proof ($C(C)\!\downarrow \iff C(C)\!\uparrow$); undecidability *by reduction*; the uncomputable $\varphi_x(x)+1$.
6. **Recursive functions:** basic functions + composition + primitive recursion = PRIM; + minimisation = partial recursive = computable; Ackermann is total computable but not p.r.